

Bar and Kitchen Inventory Management System (Powered by LIS), Online/Offline, Event-Driven Architecture

PROJECT PROPOSAL AND SYSTEM SCOPE

Prepared for:

Lourd Borromeo, Liquor Inventory Solution

Prepared by:

Bassig, Jj Sanchez

Espartero, Rasty Cannu

Date: June 24, 2026

I. Overview

Bar and Kitchen Inventory Management System is a modern inventory management system designed for bar and kitchen operations that require accurate, traceable tracking of stock movements, sales, and purchasing activities. The system is built to be flexible across three platforms: web, desktop, and mobile, to fully support online and offline forms of inventory under a single unit-of-measurement framework.

The system is being developed as a significant upgrade of the client's existing inventory system. The current system will be shared with the development team and used as a functional and structural reference. Its existing logic, data columns, and report formats inform the design of the new platform, while the new system itself is built on a modern, audit-grade architecture that resolves the legacy setup's technical limitations.

The system is built to work even without an internet connection and automatically synchronizes once connectivity is restored. This ensures that operations continue smoothly even in environments where network access is unstable or unavailable.

Unlike traditional inventory systems, Bar and Kitchen Inventory Management System records every stock movement as a permanent transaction. This ensures that all inventory history is fully traceable, transparent, and cannot be altered without proper system records, while still allowing any entry, whether manually encoded or system-recognized, to be corrected or undone without losing the original record.

II. System Purpose

The main purpose of Bar and Kitchen Inventory Management System is to solve common inventory problems such as:

- Inaccurate stock counts
- Missing or untraceable transactions
- Manual errors in recording inventory
- Difficulty tracking stock across multiple branches
- Loss of data during offline operations

The system ensures that all inventory changes are recorded properly and can always be reviewed at any point in time.

III. Scope of Work

The system is designed as a fully event-sourced inventory ledger platform, where all inventory movements are recorded as immutable events. The system does not store or mutate stock values directly. Instead, stock is computed through deterministic event replay. All operations are permanently logged with full traceability, including user, device, timestamp, and location.

For this engagement, the architecture is built to be multi-branch and multi-tenant capable, but the initial installation covers a single location with two user accounts (Owner and Employee), in line with the agreed scope, budget, and timeline. Multi-branch rollout and the client's planned resale of the system to his own clients are addressed separately under Section 4 (System Packages & Deployment Options).

1. Inventory Event Tracking (Core Ledger System)

The system records all inventory movements as structured events:

- Stock In (STOCK_IN) — receiving deliveries, restocks, or initial inventory entries
- Stock Out (STOCK_OUT) — consumption, sales, usage, or wastage
- Stock Transfer (STOCK_TRANSFER) — movement between locations or branches

- Stock Adjustment (STOCK_ADJUSTMENT) — system or audit-based corrections due to discrepancies
- Stock Revert (STOCK_REVERT) — compensating event that reverses a previously recorded incorrect transaction

Critical System Rule:

All inventory operations are append-only events.

No event may be edited or deleted after creation.

Stock values are never stored directly and are always derived from event history.

Undo and Correction

Every input, whether manually encoded, imported from a file, or recognized through the parsing/AI layer described in Section 3.3, can be undone or corrected by the user. Corrections are implemented as a new Stock Revert or Stock Adjustment event, so the original entry is preserved for audit purposes while the corrected figure becomes the active record. This applies to purchase records too: a purchase entry can be edited or deleted from the user's point of view, while the system internally records this as a compensating event rather than a silent overwrite.

2. Accountability and Immutable Audit Model

Each event is permanently tied to:

- User identity
- Device identifier
- Timestamp of creation
- Location or branch context

The system enforces strict immutability:

- Events cannot be modified
- Events cannot be deleted
- Historical data cannot be overwritten

Any correction is implemented through a new compensating event, preserving full audit integrity. This ensures a complete and tamper-proof audit trail for all inventory activities.

3. Document Import and Data Entry

Inventory events can be created through the following controlled input methods:

- Manual Entry: direct encoding of inventory events by authorized users
- Structured File Import: CSV or Excel files, including POS exports, parsed into validated events
- System-Generated Forms: built-in encoding forms for staff who do not have a source file to upload
- Document-Based Entry (Optional Phase): PDF-based supplier documents converted into structured event data after validation

Recognized Transaction Categories

Imported and encoded data is classified under four categories:

- Audit
- Sales
- Non-Sales (Non-Revenue):e.g., wastage, comps, internal use, spoilage
- Purchase

For each category, the system recognizes the following fields directly from the source file or form:

- Quantity
- Discount
- Price
- Amount

The system can then generate summarized files (Excel or CSV) on a weekly, monthly, or yearly basis from this recognized data.

Recognition Approach

The initial build uses a structured, template-based parsing engine for the common POS export and Excel layouts already in use by the client. This covers the stated requirement without the added cost and time of a full AI/OCR pipeline. If source files later prove inconsistent (e.g., scanned receipts, non-standard layouts), an AI-assisted recognition layer can be added as a future enhancement.

Validation and Correction

All imported or recognized data passes through a review and confirmation layer before being committed as events into the ledger. No automatic import is allowed without user validation. If the recognition engine misreads a value, the user can correct or undo it before or after committing, per the Undo and Correction model in Section 3.1.

4. Offline Operation and Synchronization Model

The system operates under a strict offline-first architecture.

Offline Flow:

- User creates inventory event
- Event is stored locally (SQLite)
- Event is queued in a local outbox
- Events are synchronized when connectivity is restored
- Server validates and commits events atomically

Guarantees:

- No data loss during offline operation
- No dependency on continuous connectivity
- Full recovery after device shutdown or crash
- Deterministic synchronization across devices

5. Stock Computation and Historical Reconstruction

Stock is not stored as a field in the database.

Instead, stock is computed using:

Deterministic replay of all relevant inventory events

The system supports:

- Current stock computation per product and location
- Historical stock reconstruction at any point in time
- Time-based inventory analysis (daily, weekly, monthly, yearly)

6. Audit and Variance Detection

The system automatically detects discrepancies between:

- Computed stock (event-based system state)
- Physical stock count input by users

Variance types include:

- Usage vs sales mismatch
- Overstock or shortage
- Non-revenue consumption (e.g., wastage, comps, breakage)
- Cost variance between purchase and expected value
- Pour variance against a configurable threshold (e.g., over-pour / short-pour on liquor shots)

Threshold-Based Highlighting

Thresholds can be configured per item or category (for example, an acceptable variance range for liquor pours, or an acceptable quantity variance for a given ingredient). Any reading that falls outside the configured threshold is automatically flagged and highlighted in the Detailed Audit Report, so problem areas are visible without manually scanning the full report.

Correction Model:

All discrepancies generate a Stock Adjustment Event, which preserves history while correcting derived state. No historical modification is permitted.

7. Sales Event Processing

Sales transactions are recorded as Stock Out Events, ensuring direct integration into inventory computation.

Sales data supports:

- Manual encoding of transactions
- Structured import from external files (including POS exports, per Section 3.3)
- Integration into reporting and variance analysis

Sales are not stored as separate financial records within the inventory ledger; they are treated as inventory consumption events. When a recipe is revised (Section 3.9), future sales automatically draw from the updated recipe, while past sales remain tied to the recipe version that was active at the time.

8. Purchasing and Procurement System

The purchasing module supports full procurement lifecycle management:

- Purchase order creation linked to suppliers
- Supplier contact management
- Par stock-based reorder recommendations
- Cost averaging across multiple purchase events, computed weekly, monthly, or yearly
- Purchase records can be arranged by date encoded or by cutoff period, and edited or deleted by authorized users (recorded internally as compensating events, per Section 3.1)
- Export of purchase orders and procurement summaries in Excel, CSV, and PDF format, on a weekly basis or any selected range
- Email-based purchase order dispatch

All purchase receipts are recorded as Stock In Events within the ledger system.

9. Recipe and Menu Engineering

Menu items are defined by encoding their ingredients and the quantity of each ingredient used (a recipe, or bill of materials). This recipe is what links a menu item to inventory consumption.

Ingredient Propagation

Because ingredients are shared across multiple menu items, updating a single ingredient automatically applies everywhere that ingredient is used. For example, if green bell pepper is used in 10 different menu items, a single update to that ingredient (price, unit of measurement, or substitution) is reflected in the cost and stock consumption of all 10 items, without updating each menu item individually.

Recipe Versioning

Each recipe change is preserved as its own version. If a menu item's ingredients are later revised, previously generated reports continue to reflect the recipe that was active at the time of that sale, while new sales draw from the updated recipe. This keeps historical reports accurate even after a menu change.

10. Unit of Measurement (UoM) Flexibility and Conversion

The system supports flexible unit-of-measurement handling across every inventory type (from bar glassware and bottled liquor to kitchen ingredients and fixed business assets) under a single, consistent framework.

Automatic Conversion

Items can be encoded in one unit and automatically converted to a standard base unit for stock computation, for example:

- 500 grams → 0.5 kg
- Pounds → ml (using an item-specific conversion factor, for liquids measured by weight)
- Liter → ml
- ¼ (unit) → 0.25 kg

Conversion factors are configured per item, so the same logic applies consistently across glassware counts, bottle volumes, ingredient weights, and asset units.

Liquid Weight Encoding (Open Bottle Estimation)

For bar operations, the system supports estimating the remaining liquid in an open bottle by weight. This feature can be enabled or disabled per item or category, depending on whether the client wants to track partially used bottles this way.

11. Physical Count Sheets

The system can generate count sheets for physical stock-taking, organized either by category or in alphabetical order, to make manual counting faster and easier to reconcile against computed stock. Counts entered through this sheet feed directly into the Audit and Variance Detection process described in Section 3.6.

12. Multi-Location and Branch Inventory Model (Architecture-Ready)

The underlying architecture supports tracking inventory independently across multiple locations and branches, with each location maintaining its own event stream and no shared stock pool between branches.

For this contract, the initial installation is scoped to a single location. Multi-branch activation is available as part of the packages described in Section 4, when the client is ready to expand to additional locations or onboard his own clients.

13. Dashboard and Analytics Layer

The dashboard provides computed insights derived from event replay, including:

- Real-time stock levels per product and location
- Sales and consumption trends
- Inventory alerts for low stock
- Variance and discrepancy indicators, with threshold-based highlights
- Analytical charts and summary metrics

All dashboard data is read-model based, derived from the event system.

14. Reports

The system provides a comprehensive reporting engine built on top of the event replay and read-model architecture. All reports are generated dynamically from immutable event data to ensure accuracy and audit consistency.

Reports are fully exportable in PDF, Excel, and CSV formats, with automatic formatting (column width and sizing) and highlighted variances on export.

14.1 Report Periods

Reports can be generated daily, weekly, monthly, or yearly, with a flexible custom date range, and can be aligned to the client's defined cutoff period rather than only the calendar month.

14.2 Standard Reports

The system includes the following core reports:

- Full Audit Report (complete event history across modules)
- Date-Filtered Audit Report (daily, weekly, monthly, yearly views), with automatically highlighted problem areas (over/short variances against threshold)
- Sales Report (derived from STOCK_OUT events classified as sales)
- Average Sales Report
- Purchase Report (derived from STOCK_IN events linked to suppliers), with weekly/monthly/yearly cost averaging
- Inventory on Hand Report (current computed stock per product and location, with cost price and selling price)
- Cost Analysis Report (purchase cost aggregation from event history)
- Non-Revenue Report (wastage, comps, internal use)
- Beverage Cost Report
- Asset Management Report (quantity, current price, market price)
- Full Detailed Report by Category

The client will provide additional specific report requirements to be pulled from the above as the build progresses; the reporting engine is designed to support additional report definitions without re-architecture.

14.3 Custom Reports

The system supports configurable custom reports, allowing filtering and aggregation based on:

- Product
- Location / Branch
- Time range / cutoff period
- Event type
- User activity

Custom reports are generated from the same event ledger, ensuring consistency with system-wide calculations.

14.4 Report Generation Model

All reports are generated using: Deterministic event replay + cached snapshot optimization

This ensures:

- No manual data manipulation
- No stored "report tables" as source of truth
- Full historical traceability

14.5 Performance Strategy

To ensure scalability, the system uses:

- Snapshot caching for frequent queries
- Indexed event queries (product_id, timestamp)
- Incremental recalculation for large datasets

This prevents full replay computation on every request while maintaining accuracy.

15. Master Data Management

Master Data defines the core reference entities used across the system. These entities are static or semi-static and are referenced by inventory events.

15.1 Items (Products)

- Product name and description
- Unit of measurement, with configured conversion factors (Section 3.10)
- Cost price (for valuation)
- Selling price (for sales analysis)
- Category assignment
- Asset classification, for items tracked as business assets rather than consumable stock

15.2 Categories

- Logical grouping of inventory items
- Used for reporting, filtering, and physical count sheet organization

15.3 Suppliers

- Supplier name and contact details
- Used for procurement tracking and purchase order association

15.4 Recipes

- Ingredient list and quantity per menu item, with versioning (Section 3.9)

Key Principle

Master data does NOT store inventory movement.

It only provides reference context for event-driven inventory transactions.

16. User Roles and Access Control

The system implements a role-based access control (RBAC) system with module-level permissions.

16.1 Roles

For this installation, two active user accounts are provisioned:

- Owner (Client Admin)
- Full access within the installation
- User management, configuration, and reporting
- Can edit any record, depending on authorization rules
- Employee (Staff)
- Limited operational access
- Event creation based on assigned modules only

A third role, Global Admin, exists at the architecture level to support future multi-location or multi-client management (Section 4) but is not an active user account under this single-installation contract.

16.2 Permission Model

Permissions are defined at:

- Module level (Inventory, Sales, Purchasing, Reports)
- Action level (Create, View, Export, Adjust, Edit)

All fields and configurations are editable by the user or admin, depending on their assigned access or authorization level. This ensures granular control over system operations.

16.3 Credentials

The system generates a username and password for each new user account created by the Owner/Admin.

16.4 Audit Enforcement

Every user action is logged as an immutable event, ensuring:

- Full accountability
- Device traceability
- Historical reconstruction of all system actions

17. Audit Trail

The system maintains a complete immutable audit trail of all activities through its event-sourced architecture.

Every system interaction is recorded as an event containing:

- User ID
- Device ID
- Timestamp
- Location or branch
- Event type
- Data payload

Key Properties

- Audit logs are append-only
- No deletion or modification is allowed
- Audit history is globally consistent across all modules
- Events serve as both operational data and audit records

Audit Capability

The system supports full reconstruction of:

- User activity history
- Inventory movements
- Sales and purchasing history
- Discrepancy resolution history

18. Desktop Application

For this contract, the system is delivered as a desktop application, installed on one (1) client computer.

- Built using Electron
- Supports Windows and macOS
- Primary and sole operational interface for this installation
- Full offline capability with local SQLite storage

A mobile application is designed into the broader architecture but are parked for this phase. See Section 3.21.

19. User Interface and Experience

The system features a modern, operationally optimized interface designed for high-frequency inventory environments such as bars, kitchens, and storage facilities, using a blue-themed color palette per the client's preference.

Design Principles:

- Minimal input friction for staff efficiency
- Fast event creation workflows
- Clear visibility of stock and movement history
- Role-based UI simplification
- Blue motif, consistent across all screens

UX Objective

The interface is designed not as a traditional dashboard system, but as an event capture system, ensuring:

- Fast recording of inventory movements
- Reduced manual errors
- Immediate feedback on system state

The development team will continue to propose UX and workflow improvements on top of the client's existing logic, to keep day-to-day use straightforward for staff.

20. Security Features

The client asked us to add the security features previously discussed. Listed below are the standard items that fit the system as scoped. *Replace or add to this list with the specific points from your actual discussion with him:*

- Role-based access control limiting each user to their authorized modules and actions (Section 3.16)
- Append-only, tamper-evident audit trail (Sections 3.2, 3.17)
- License-bound installation, so the application isn't copied or run on unauthorized machines
- Subscription-based database access scoped to authorization level, for the future packages in Section 4
- System-generated credentials for each user account

21. Minor Enhancements

The following are included as minor enhancements within the current scope:

- Forfeited stock encoding: recording stock that is forfeited (e.g., expired, lost, spoiled) as a distinct, reportable transaction type
- Liquid weight encoding toggle (Section 3.10)
- Undo capability across all manual and recognized inputs (Section 3.1)

22. Parked / Future Enhancements

The following were discussed but are explicitly excluded from the current ₱50,000 / 2-month contract. They are compatible with the architecture and can be scoped as separate future engagements:

- Mobile application UI (parked)
- AI/OCR-based recognition for inconsistent or scanned source documents (Section 3.3)
- Multi-branch rollout beyond the single installation covered here

23. Out of Scope

The following features are not included in the current system implementation:

- Accounting or financial bookkeeping systems
- Browser-based web application interface (a web-hosted backend/database, per Section 4, is supported: this refers specifically to a browser-based user interface)
- Mobile application UI

24. Transition from Existing System

The client's existing system will be shared with the development team and used as a reference for current logic, data structure, and reporting needs. The new system will not perform a live data migration of historical records; rather, it adopts the relevant logic and structure into the new architecture.

Transition Policy:

- The new system will operate on a clean-slate data model
- No historical transactional data migration will be performed
- The existing system's logic and structure are used as a reference during development, not as the live source of truth
- All inventory records will begin from system activation date

Rationale:

This approach ensures:

- Data integrity under the new event-sourced architecture
 - Elimination of legacy inconsistencies
 - Clean audit trail from system start date
4. System Packages and Deployment Options

The client intends to apply the system in a single installation and subscription model, with System Package tiers (Basic, Medium, and Full) distinguishing the level of access and functionality available. Exact feature boundaries and pricing per package are the client's commercial decision and are not defined by this proposal.

Deployment options:

- On-premise installation: Electron desktop application installed on a client computer (Section 3.18)
- Web-hosted: the system uploaded to web hosting, with database access for each user scoped by subscription tier and authorization level (Section 3.20)

5. System Requirements (For Installation)

Per the client's request, if he proceeds with on-premise installation, the development team will provide the computer software and hardware requirements for the client's machine. In general, this kind of desktop application (Electron + local SQLite) runs comfortably on a standard mid-range Windows or macOS computer. The exact minimum specs will be confirmed once the legacy system and target machine are reviewed at project start.

6. Documentation, Training, and Support

The following are included as part of system delivery:

- User manual covering day-to-day operation of the system
- System-generated usernames and passwords for each user account (Section 3.16)
- A brief training session for the Owner and Employee accounts
- Remote/online delivery of updates for bug fixes and minor enhancements after go-live
- Flexibility for minor adjustments to the system based on continued client feedback

The system remains under the sole administrative control of the client (Liquor Inventory Solution) or staff the client designates; the development team's role after deployment is limited to the support, debugging, and minor-enhancement scope described above, unless otherwise agreed.

IV. Stages of Work

Stage	Description
Data Gathering	Requirements collection and system specification alignment
Environment Setup	Infrastructure, database, and development environment configuration
Development	Full system implementation based on approved architecture
Testing and Revision	Functional testing, bug fixing, and client feedback integration
Implementation	User training and documentation delivery
Deployment	Production release across all platforms
Support and Maintenance	Ongoing bug fixes, minor enhancements, and operational support

V. Project Timeline

Stage	Description
Data Gathering & Legacy System Review	1 week
Environment Setup	1 week
Development Phase 1 (Core backend, authentication, database, master data, recipe engine)	2 weeks
Development Phase 2 (Inventory, sales, purchasing, reports, audit/variance, UoM conversion)	2 weeks
Testing and Revision	1 week
Implementation, Training & Deployment	1 week
Total Estimated Duration	8 weeks (2 months maximum)

This timeline assumes the client's existing system and reference materials (codes, sample files, reporting formats) are made available at the start of the Data Gathering stage.

III. Technology Stack

The system requires a modern architecture optimized for **offline-first, event-sourced, distributed inventory systems**.

Layer	Technology	Justification
Backend	NestJS (TypeScript)	Structured, modular, supports event-driven architecture
Database	PostgreSQL	Strong transactional integrity and audit performance
Desktop App	Electron	Native offline capability with local persistence
Mobile App	React Native	Cross-platform deployment for field operations
Offline Storage	SQLite	Reliable embedded local event storage

Architectural Justification

The originally requested stack (PHP Laravel + MySQL) is optimized for traditional CRUD systems. However, Bar and Kitchen Inventory Management System requires:

- event sourcing
- offline-first operation
- distributed synchronization
- deterministic replay computation

These requirements are not natively supported by Laravel-based architectures without significant custom engineering overhead.

10. Deployment Strategy

The Bar and Kitchen Inventory Management System backend is hosted on a NestJS API server secured over HTTPS, connected to a PostgreSQL database. For this contract, client deployment is a single desktop channel: an Electron application installed on the client's designated computer. The mobile channel remains parked per Section 3.22.

Offline operations are handled through local SQLite storage, with events queued in an outbox and synchronized to the backend when connectivity is available; the backend validates and commits events atomically.

Backend updates, bug fixes, and minor enhancements are deployed remotely, allowing the system to be updated online without an on-site visit, per Section 6.

11. Project Cost

The Bar and Kitchen Inventory Management System is a custom-built inventory management solution designed for offline-first operation with audit-grade data integrity, built by substantially upgrading the client's existing inventory process into a modern event-sourced architecture.

The agreed project investment for this scope covers the design, development, testing, deployment, implementation, documentation, and onboarding of the system described in this proposal, including inventory management, purchasing, sales processing, audit and variance reporting, recipe management, unit-of-measurement conversion, reporting, user management, and future-ready subscription architecture

The mode of payment for the developers' compensation would be made through bank transfer between the client and the project manager.

Special Startup Implementation Rate

As part of this engagement, the development team is extending a heavily discounted startup implementation rate.

The system is being developed with a scalable architecture that supports package-based deployment models, allowing clients to manage varying numbers of restaurant branches and user accounts depending on their subscribed package. The platform foundation is designed to support subscription-based services, client onboarding workflows, role-based access control, and additional deployments without requiring a complete system rebuild.

The agreed project fee should therefore be viewed as a discounted implementation investment for the initial deployment and not as the full commercial value of the platform being developed. The pricing reflects the startup-stage nature of the engagement while establishing a foundation that can accommodate future growth, additional locations, and expanded functionality.

Agreed Project Investmentz

₱54,000 PHP

Payment Breakdown

Milestone	Amount
Project Mobilization and Initial Development	₱20,000
Core Features Completion	₱15,000
Final Deployment and Project Completion	₱15,000
Development Resources, Infrastructure, and Third-Party Services	₱4,000
Total Project Cost	₱54,000

Included in the Investment

Core Features

- Legacy system review and process analysis for an improved modern framework
- System architecture and database design
- Inventory ledger and audit engine
- Purchasing and supplier management
- Sales and inventory consumption tracking
- Recipe and menu management
- Unit-of-measurement conversion framework
- Audit and variance detection
- Reporting engine (Excel, CSV, PDF)
- User roles and authorization controls

Operational Features

- Documentation and user manual
- Initial user onboarding and training
- Deployment assistance
- Initial debugging
- Multi-tenant deployment
- Subscription-based access management
- Additional client onboarding
- Multi-branch operations
- Desktop application packaging

Future Enhancements & Prototyping

While the mobile application and comprehensive AI integration are primarily slated as future enhancements, the development team is committed to delivering initial proof-of-concept (PoC) prototypes for both, as previously discussed.

The AI prototyping will focus on two key areas: an assistant chatbot and the AI-powered ingestion of Point of Sale (POS) data. The chatbot will utilize a custom Retrieval-Augmented Generation (RAG) architecture to ensure accurate, context-aware responses. Concurrently, the POS data ingestion will be securely routed through APIs from industry-leading frontier models. The developers will implement strict safety guardrails to prevent AI hallucination or system misuse. Please note that third-party API services will operate on a pay-as-you-go model, with costs shouldered by the client.

Maintenance & IT Support

Post-deployment IT support will be continuous, featuring scheduled monthly check-ins to monitor system health. Urgent technical issues will be treated with high priority and addressed as quickly as possible upon being reported. Ongoing support and maintenance requests will be billed accordingly based on agreed-upon developer rates.

Terms and Conditions of Use

Bar and Kitchen Inventory Management System is provided as an offline-first, event-sourced inventory management system designed for tracking inventory movements, sales, purchases, and related operational data. By using the system, users acknowledge that all inventory records are generated through user-created events, and the system functions strictly as a ledger-based tracking tool rather than a financial, accounting, or tax reporting system. The accuracy of inventory data depends on the correctness of the events entered or recognized by authorized users, and the system does not independently validate the business correctness of such inputs beyond structural and technical validation.

All data generated within the system is stored as immutable events, meaning once an inventory transaction is recorded, it cannot be modified or deleted outright. Any corrections must be made through compensating or reversal events in accordance with the system's event-sourcing design, and remain visible to the user as an "undo" or "edit" action even though the underlying record is preserved. The system operates in both online and offline modes, and while synchronization is designed to be reliable and atomic, temporary inconsistencies may occur during offline operation until full synchronization is completed. Users acknowledge that complete system consistency is only guaranteed after successful synchronization.

The system remains under the sole administrative control of the client or staff the client designates. The system provider is not liable for losses arising from incorrect data entry, device failure, unsynchronized offline data, or misuse of system functionalities. Users are responsible for ensuring proper device operation, secure access control, and accurate encoding of inventory events. The system is intended solely for operational inventory management and does not replace accounting systems, tax systems, or legal financial reporting tools. Any future resale of the system to the client's own clients is subject to a separate agreement, as noted in Section 4.

13. Usage Policy

Bar and Kitchen Inventory Management System must be used strictly for legitimate inventory management purposes, including recording stock movements, sales transactions, purchasing activities, reporting, and inventory reconciliation. Users are required to ensure that all entries reflect true operational events, as the system enforces an immutable event history where all actions are permanently recorded and traceable. Any attempt to manipulate, falsify, or bypass event logging, synchronization processes, or audit trails is strictly prohibited.

Users are not allowed to modify historical records, interfere with system synchronization, inject unauthorized data into the event stream, or attempt to copy, reverse-engineer, or compromise the integrity of the licensed installation. The system is designed to maintain full audit transparency, and any violation of these principles may result in restricted access or termination of system privileges.

The system may operate offline, but users remain responsible for ensuring that devices used for data entry are secure, functional, and properly synchronized when connectivity is restored. Bar and Kitchen Inventory Management System is intended exclusively for inventory tracking and operational reporting and must not be used for financial manipulation, fraudulent activity, or any form of misrepresentation of business records.